Software Test Plan (STP)

Smart Traffic Complaint and Monitoring System

Project team: Arya, Ansh, Arnav, Avyukt

Technology baseline: MERN + Python FastAPI / PyTorch inference service

Document status: Updated to reflect the implemented image-upload and AI-classification integration, and the observed GitHub Actions CI run on 30 October 2026. Manual test cases remain Not Run unless separately executed and evidenced.

1. Introduction

1.1 Purpose

This plan defines the scope, approach, environment, responsibilities, entry/exit criteria, and test cases for validating the Smart Traffic Complaint and Monitoring System against its SRS. It also records automated CI evidence and the additional test coverage needed for image upload and AI-assisted road classification.

1.2 Scope

Testing covers account access, complaint submission and retrieval, administrator workflows, validation, authorization, API errors, basic performance, browser compatibility, optional image upload, and integration with the FastAPI / ResNet18 classifier.

2. Test Strategy

• Functional testing: verify behavior against the existing FR-01 through FR-15 requirements.
• Integration testing: verify React UI, Express API, MongoDB interactions, uploaded-image handling, and the Express-to-FastAPI inference request.
• Security testing: verify authentication, ownership checks, role restrictions, file type/size validation, input validation, and safe error responses.
• Non-functional testing: execute performance, usability, availability, and browser checks where the required environment is available.
• Regression testing: rerun relevant cases after changes.
• CI testing: GitHub Actions runs the backend Jest test suite and frontend production build on pushes and pull requests targeting main.

3. Test Items and Features to be Tested

• Registration, login, logout.
• Complaint creation, required-field and category validation.
• Citizen complaint list and status tracking.
• Administrator list, filtering, priority, assignment, status update, dashboard.
• Authentication, role-based authorization, ownership enforcement.
• Optional JPG/PNG complaint image upload, file-size/type validation, and image retrieval.
• AI prediction request, category/confidence storage and display, and graceful handling when the inference service is unavailable.
• API error responses, performance, compatibility, and usability.

4. Features Not to be Tested in This Release

• Live integrations with municipal traffic authority systems.
• Real-time traffic sensor feeds.
• Live map integrations.
• Production deployment and external municipal workflow integration.
• AI prediction accuracy as a clinical/road-authority guarantee; the model output is an assistance signal and requires human review. Model quality benchmarking on a separately curated holdout dataset is outside the CI workflow.

5. Test Approach, Environment, and Criteria

5.1 Test Environment

• Application: React/Vite frontend, Node.js/Express API, MongoDB database.
• AI service: Python FastAPI, PyTorch ResNet18 model, local model weights; MPS was observed on the developer's Mac during a local health check.
• CI environment: GitHub Actions hosted runner, Node.js 24, npm dependency installation via npm ci.
• Browsers: current stable Chrome and Firefox (manual compatibility checks remain to be recorded).
• Test data: dedicated test users (at least one citizen and one administrator) and seeded complaints.
• Do not run destructive or load tests against production data.

5.2 Entry Criteria

• A runnable build is available in a test environment.
• Test accounts and test data are prepared.
• SRS requirements and allowed statuses/priorities are agreed by the team.
• For AI integration tests, the FastAPI service and model weights are available.

5.3 Exit Criteria

• All planned cases have an execution status recorded as Pass, Fail, Blocked, or Not Run.
• No unresolved critical security or data-loss defects remain.
• Failed cases have defect notes and retest status.
• Results are reported honestly; unexecuted cases remain Not Run.
• CI jobs pass for the release commit; this does not replace manual, browser, performance, or model evaluation.

5.4 Pass/Fail Rules

A test passes only when the observed behavior matches the expected result. A test fails when the observed behavior differs. Use Not Run before execution and Blocked when a prerequisite prevents execution.

6. Test Cases

The original test cases are retained. Their execution status is Not Run because the CI run does not provide individual manual-case evidence. Automated tests are summarized separately in Section 8.

ID	Test	Requirement(s)	Steps / input	Expected result	Execution status
TC-01	Registration	FR-01	Register with valid name, unique email, and valid password.	Account created; no plaintext password stored.	Not Run
TC-02	Duplicate registration	FR-01	Register again with an email already in use.	Request rejected with clear validation/conflict message.	Not Run
TC-03	Login/logout	FR-02, SR-03	Log in with valid credentials, then log out.	Login succeeds; logout behaves as designed.	Not Run
TC-04	Invalid login	FR-02	Submit incorrect password.	Access denied; no sensitive details disclosed.	Not Run
TC-05	Valid complaint submission	FR-03–FR-05	Submit category, description and location while authenticated.	Complaint saved with unique ID, timestamp and Submitted status.	Not Run
TC-06	Missing required field	FR-14	Submit complaint with location or description missing.	400 validation response; no complaint saved.	Not Run
TC-07	Citizen complaint list	FR-06, FR-07	Log in as citizen and open complaint list.	Only that citizen's complaints and statuses are shown.	Not Run
TC-08	Cross-user access	SR-01, NFR-09	Attempt to retrieve another citizen's complaint ID.	Private complaint data is not returned.	Not Run
TC-09	Admin-only endpoint	FR-15, SR-02	Call admin complaint endpoint as a citizen.	403 Forbidden; no admin data returned.	Not Run
TC-10	Filtering	FR-08, FR-09	Filter administrator complaint list by status and category.	Only matching records returned.	Not Run
TC-11	Priority and assignment	FR-10, FR-11	Admin sets allowed priority and assigns a complaint.	Updated values are saved and shown.	Not Run
TC-12	Status update	FR-12	Admin updates status to an allowed value.	Status changes and is visible in complaint detail/list.	Not Run
TC-13	Invalid status	FR-12, SR-04	Submit status value outside allowed list.	Request rejected; existing status unchanged.	Not Run
TC-14	Dashboard counts	FR-13	Compare dashboard totals with known seeded records.	Counts match test dataset.	Not Run
TC-15	Performance	NFR-01	With up to 1,000 complaints, measure 100 list/detail requests.	At least 95% complete within 3 seconds in recorded environment.	Not Run
TC-16	Browser compatibility	NFR-04	Run core flows in current stable Chrome and Firefox.	Core flows operate without blocking browser-specific defects.	Not Run
TC-17	Usability	NFR-02	Ask five first-time users to submit a valid complaint.	At least four finish within 3 minutes after login.	Not Run
TC-18	Controlled errors	NFR-06, SR-06	Cause controlled validation or test-environment server error.	No false success; safe, documented error response.	Not Run
TC-19	Invalid complaint category	Validation	Submit a category not in the allowed category list.	API returns HTTP 400; no complaint is created.	Pass (CI retest)
TC-20	Image upload — valid file	Image upload	Submit complaint with a valid JPG/PNG image within 10 MB.	Image is stored; complaint returns an image URL.	Not Run
TC-21	Image upload — invalid type	Image upload / security	Upload a non-JPG/PNG file.	Upload is rejected with a controlled error; no file is accepted.	Not Run
TC-22	Image upload — size limit	Image upload / security	Upload an image larger than 10 MB.	Upload is rejected; no complaint/file is created.	Not Run
TC-23	AI classification success	AI integration	Submit complaint with image while FastAPI model service is available.	Prediction category, confidence and probabilities are saved and returned.	Not Run
TC-24	AI service unavailable	AI integration / resilience	Submit complaint image while FastAPI is unavailable or times out.	Complaint is still saved; AI status is recorded as Failed.	Not Run
TC-25	AI prediction display	AI integration	Open citizen/admin complaint view containing AI result.	Stored category and confidence are displayed accurately.	Not Run

ID	Test	Requirement(s)	Steps / input	Expected result	Execution status
TC-26	CI workflow	Build verification	Push to main or open PR targeting main.	Backend Jest suite and frontend production build run; result is visible in Actions.	Pass (30 Oct 2026)

7. Requirement Traceability

SRS requirement(s)	Verification test(s)
FR-01–02	TC-01–04
FR-03–05	TC-05–06, TC-20
FR-06–07	TC-07–08, TC-25
FR-08–09	TC-10
FR-10–12	TC-11–13
FR-13	TC-14
FR-14	TC-06, TC-18, TC-19
FR-15	TC-09
NFR-01	TC-15
NFR-02	TC-17
NFR-03	Availability acceptance run; Not Run until executed
NFR-04	TC-16
NFR-06	TC-18
NFR-07	TC-05, TC-14
NFR-08	TC-01, TC-03, SEC-04
NFR-09	TC-08, TC-09, SEC-01–SEC-03
NFR-10	TC-06, TC-13, SEC-05, SEC-08
Image upload / AI integration	TC-20–TC-25
CI build verification	TC-26

8. Automated Test Execution and CI Evidence

Execution date: 30 October 2026
Platform: GitHub Actions — workflow name: TrafficCare CI
Trigger: Push to main
Commit shown in Actions: 05e29a6 (workflow run titled “Validate complaint categories in backend”).

Observed result

Check	Observed result
Backend Tests	Passed in the latest workflow run after backend category validation was corrected.
Frontend Build	Passed in the latest workflow run.
Earlier backend run	39 passed, 1 failed out of 40 tests. The failing test expected HTTP 400 for an invalid complaint category but received HTTP 201.
Retest outcome	The category-validation change was pushed; the subsequent GitHub Actions run showed a green overall result. The previous invalid-category test failure was therefore resolved in CI.

Important interpretation

The green workflow verifies the automated Jest suite and frontend build only. It does not prove that all 26 test cases above have been manually executed. TC-19 is marked Pass (CI retest) based on the corrected invalid-category test and successful subsequent CI run. TC-26 is marked Pass based on the visible successful workflow. The image upload, AI end-to-end, performance, usability, browser and availability cases remain Not Run until executed with recorded evidence.

9. Known Limitations and Deviations

• The implementation scope was extended beyond the original MERN baseline to include image upload and a separate FastAPI / PyTorch ResNet18 inference service.
• The original test plan listed photo upload and AI classification among features not to be tested in the initial release. They have since been implemented as project features; dedicated end-to-end test cases have been added but remain Not Run.
• CI currently covers backend Jest tests and frontend build. It does not start MongoDB, run the FastAPI model, perform browser automation, benchmark model accuracy, or deploy the application.
• Uploaded files and local model weights/environment are runtime assets; secrets and local environment files should not be committed to the repository.

10. Roles and Deliverables

- Test lead: coordinate execution and consolidate results.
- Developers: fix defects and provide test builds.
- All team members: review requirements, execute assigned cases, and record evidence.
- Deliverables: this plan, test execution record, defect list, CI workflow evidence, and final test summary.